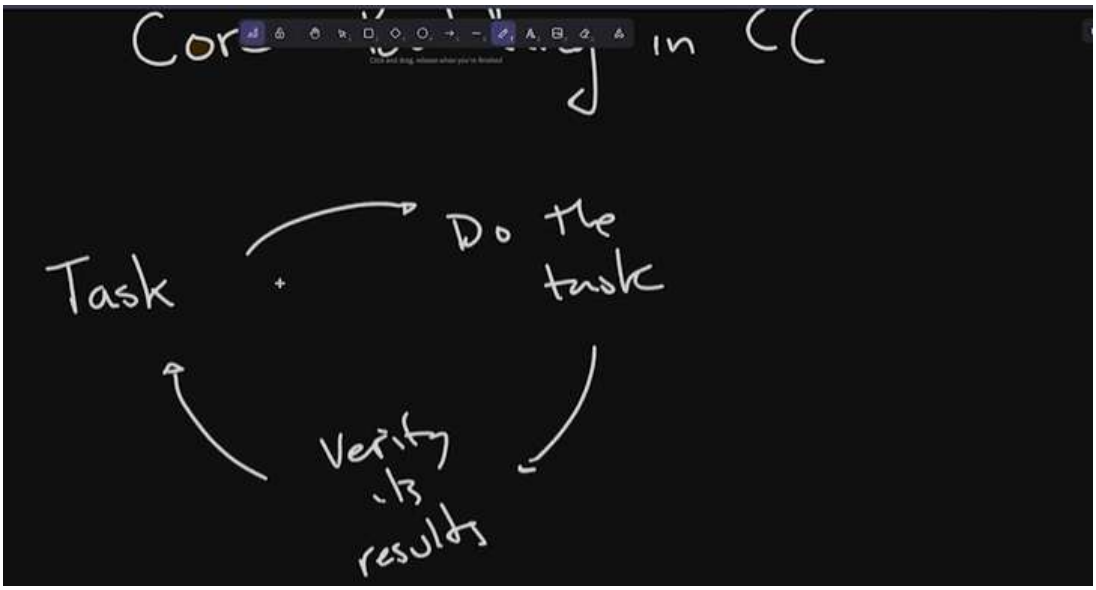
AI Agents Full Course 2026: Masre Agentic AI (2Hours)




```
~/.claude/                          # PERSONAL (global)
├── settings.json                   # Global permissions, env vars, hooks
├── CLAUDE.md                       # Your personal instructions (always
loaded)
├── keybindings.json                # Custom keyboard shortcuts
├── agents/                         # Personal subagent definitions
├── skills/                         # Personal slash commands
├── rules/                          # Rules applied to all projects
└── projects/                       # Session transcripts & auto-memory
    └── <project-hash>/
        ├── session-id.jsonl        # Full conversation logs
        └── memory/
            └── MEMORY.md           # Persistent learnings (first 200
lines auto-loaded)

.claude/                            # PROJECT (team-shared)
├── settings.json                   # Team permissions & hooks (committed)
├── settings.local.json             # Your personal overrides (gitignored)
├── CLAUDE.md                       # Project instructions (committed)
├── CLAUDE.local.md                 # Your personal project notes
(gitignored)
├── agents/                         # Project subagents
├── skills/                         # Project slash commands
├── rules/                          # Modular instruction files
│   ├── code-style.md
│   └── frontend/react.md
└── .mcp.json                       # Project MCP servers
```

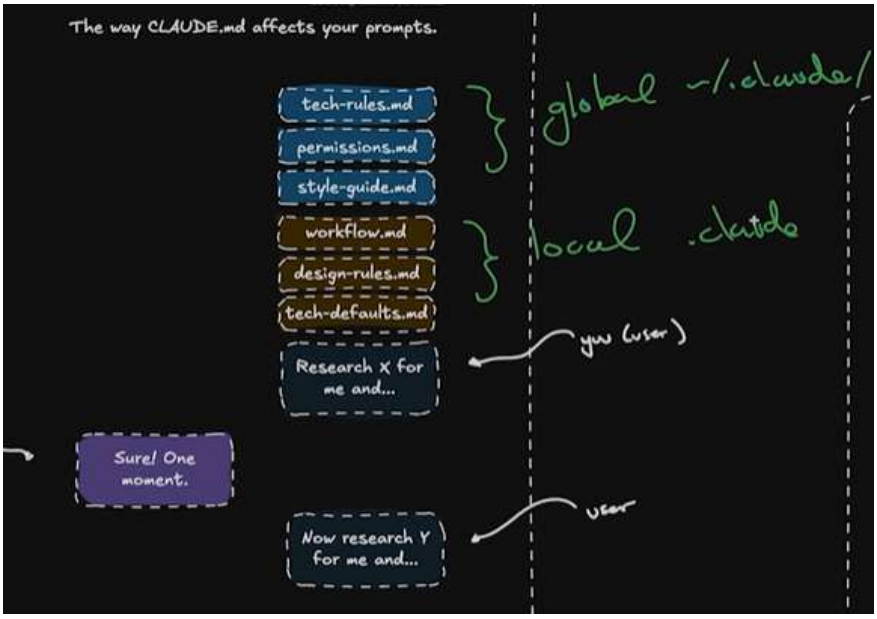
Scoped rules in `.claude/rules/`

Break instructions into focused topic files instead of one monolithic CLAUDE.md.

```
.claude/rules/
├── code-style.md
├── testing.md
├── security.md
└── frontend/
    ├── react.md
    └── styles.md
```

3 layers of CLAUDE.md that merge together

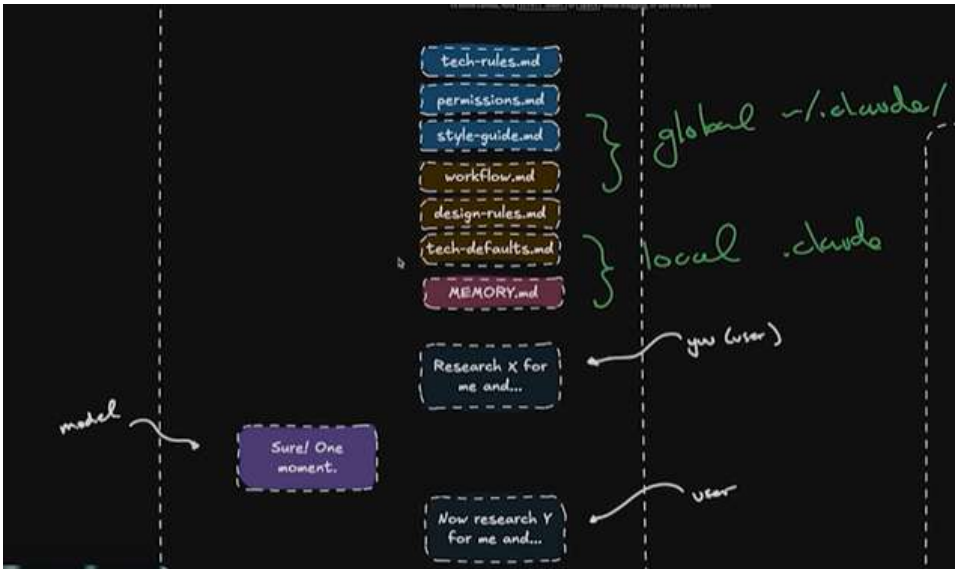
1. `~/.claude/CLAUDE.md` — personal global
2. `.claude/CLAUDE.md` — project, committed
3. Managed system-level CLAUDE.md — enterprise

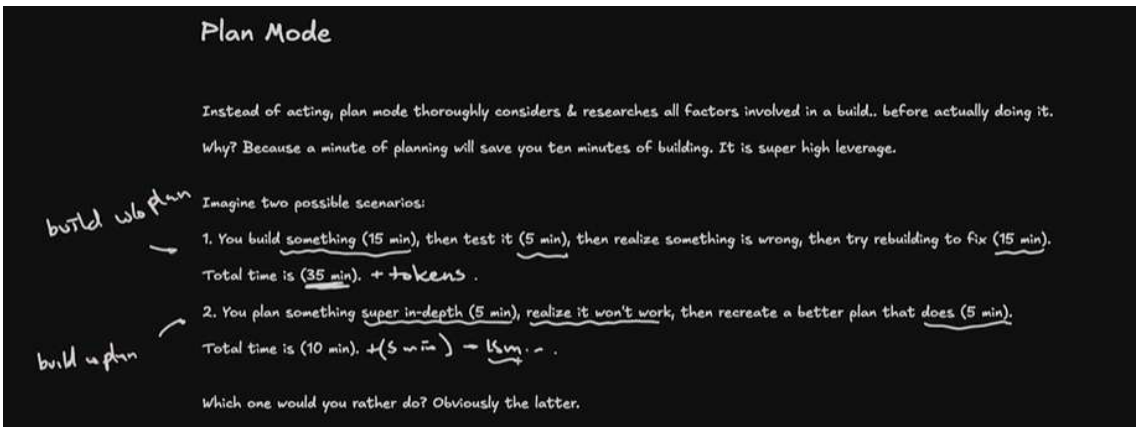


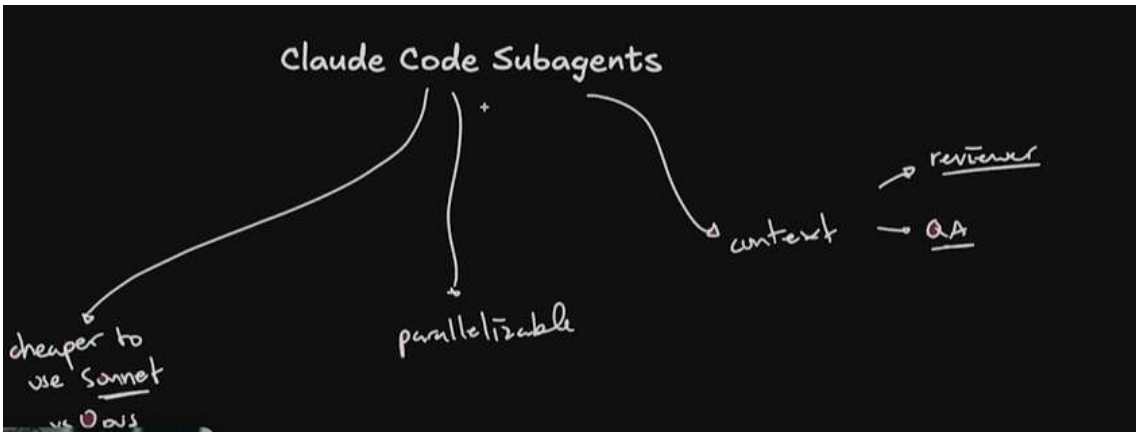
5.3 — Auto-Memory: Claude Remembers Across Sessions

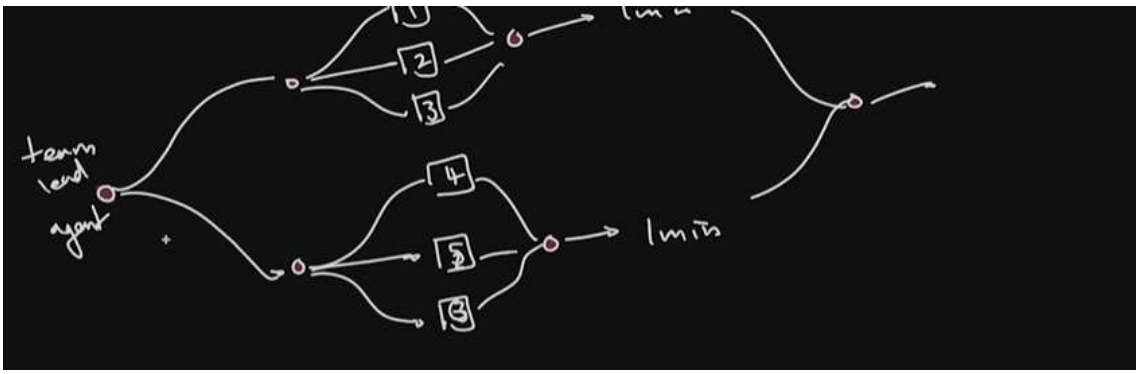
This is the sleeper feature most people miss.

- **Location:** `~/.claude/projects/<project-hash>/memory/MEMORY.md`
- First 200 lines loaded into system prompt at session start
- Claude reads and writes during the session — records debugging insights, patterns, preferences
- You can create topic files (`debugging.md`, `patterns.md`) and link from MEMORY.md
- Separate from CLAUDE.md — this is Claude's own notes, not your instructions





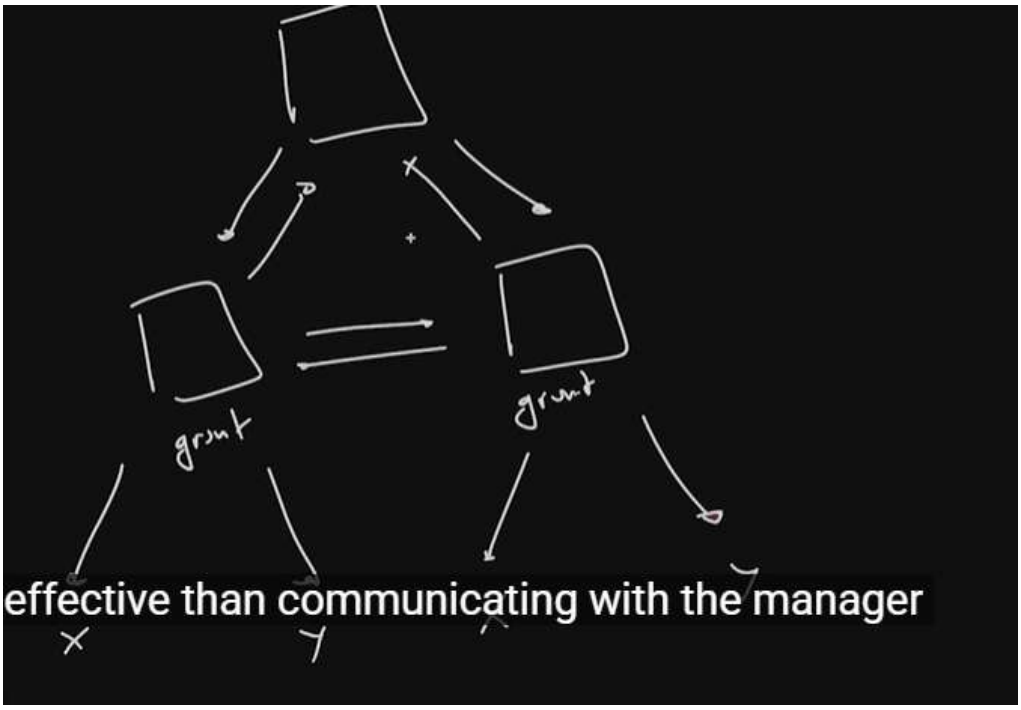


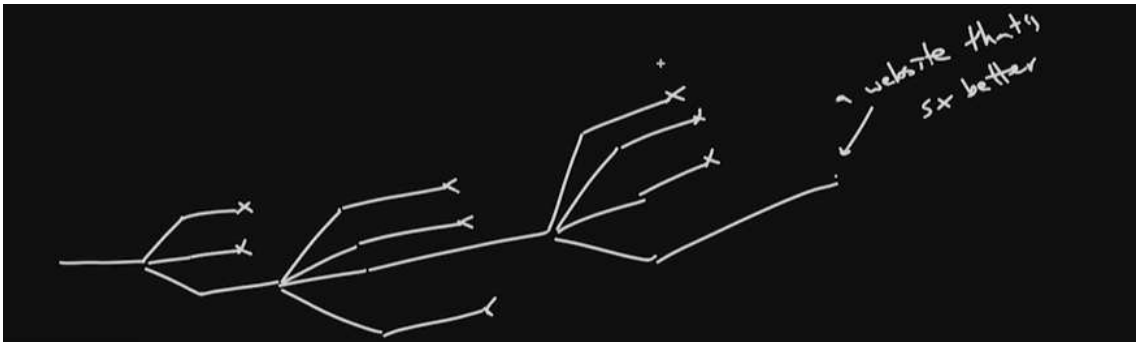


Compare with subagents

Both agent teams and **subagents** let you parallelize work, but they operate differently. Choose based on whether your workers need to communicate with each other:

	Subagents	Agent teams
Context	Own context window; results return to the caller	Own context window; fully independent
Communication	Report results back to the main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list with self-coordination
Best for	Focused tasks where only the result matters	Complex work requiring discussion and collaboration
Token cost	Lower: results summarized back to main context	Higher: each teammate is a separate Claude instance


grunt
grunt
effective than communicating with the manager

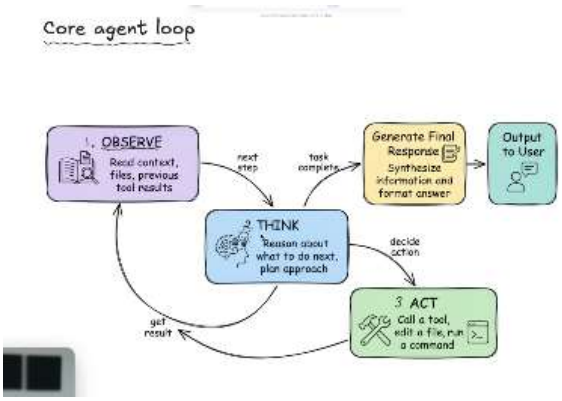

a website that's
5x better

Quick Do's & Don'ts

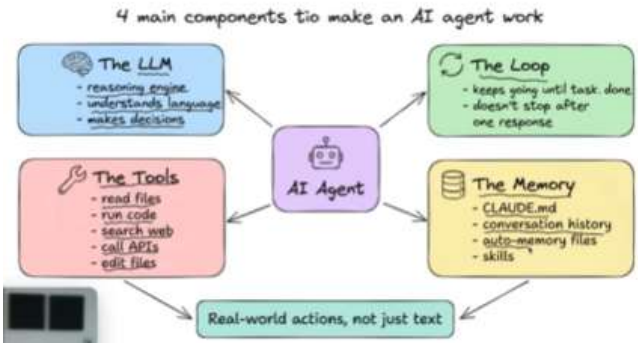
Do
- Run /init first
- Use bullet points & short headings
- Put most important guardrails at the top
- Version-control the root CLAUDE.md
- Periodically review & prune (treat like living code)

Don't
- Dump entire style guides / API docs into it
- @-include huge files unless absolutely necessary
- Write vague / aspirational rules
- Make it > 500 lines without splitting
- Forget to add rules when Claude keeps making the same mistake

AI Agent Full Course: 2026

1. Five agents controlling their own browsers
2. The core agent decision loop (OTA)
    a. Loop 3 major function
        i. 

    b. Model reach a point of definition of done, it reach task complete
    c. It output to user
    d. 4 components tio make an AI Agent work

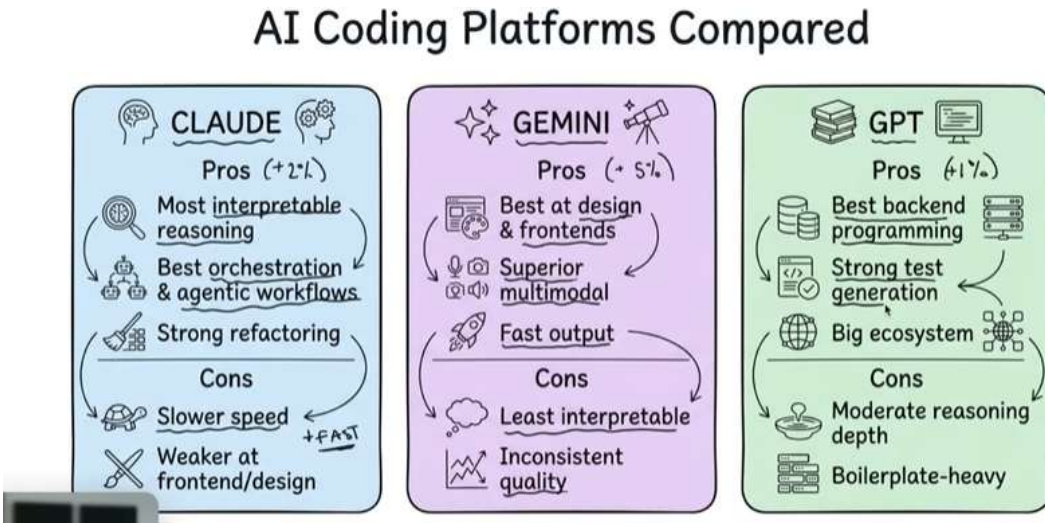
i. 

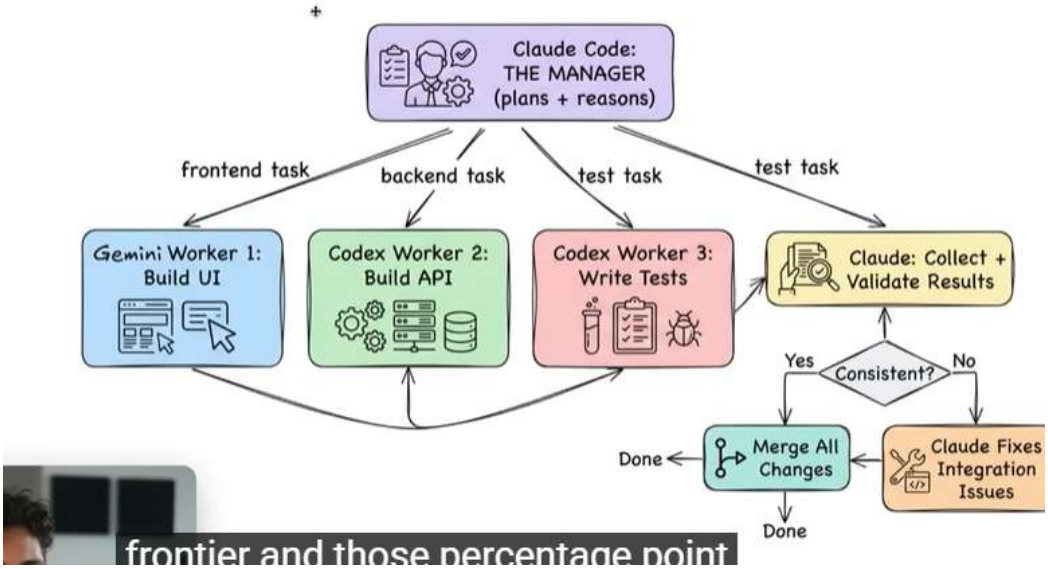
ii. Example of typical agent conversation (Codex)

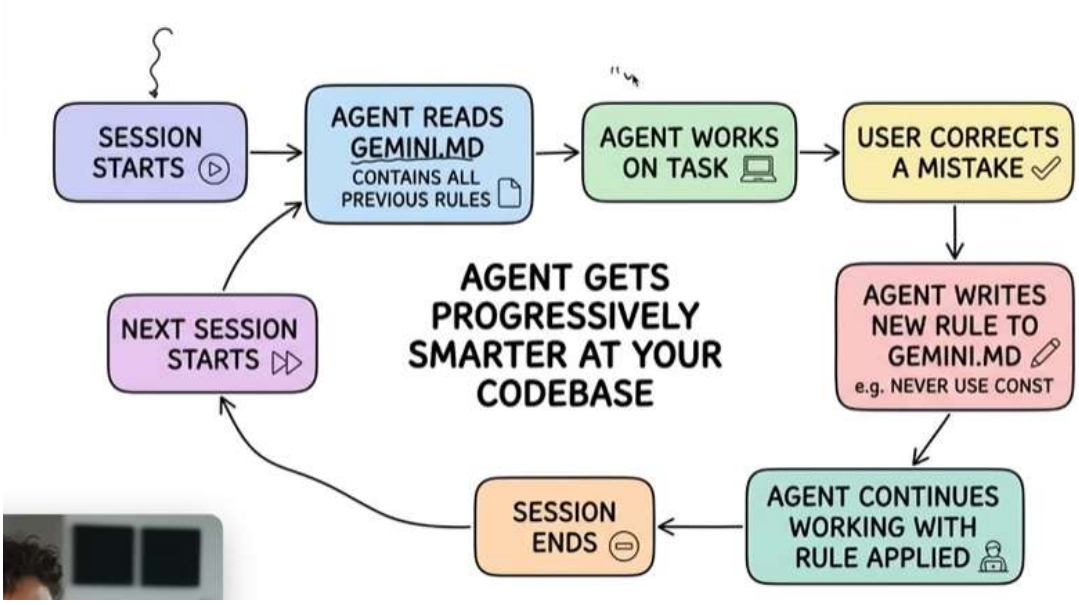
1. It is observing
2. Thinking
3. Acting

3. The major players (Codex, Antigravity, Claude Code)

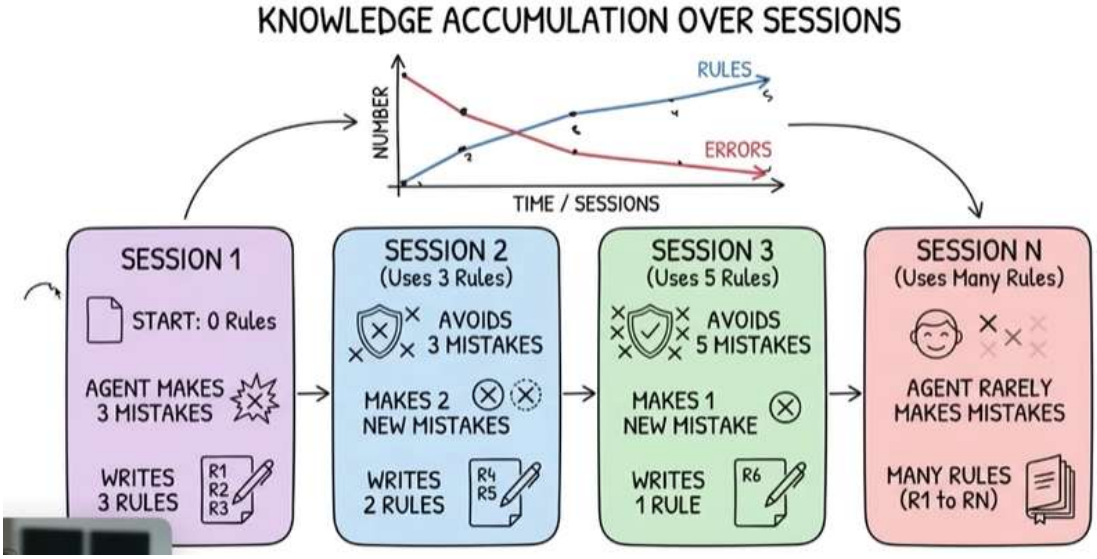
1. Chatbot is LLM
2. Agent is compination of alll
3. The LLM
    a. Reasoning engine
    b. Understands language
    c. Make decisions
4. The Tools
5. The Loop
6. The Memory
4. The major players (Codex, Antigravity, Claude Code)
5. Which AI agent to use when & why?

a. 

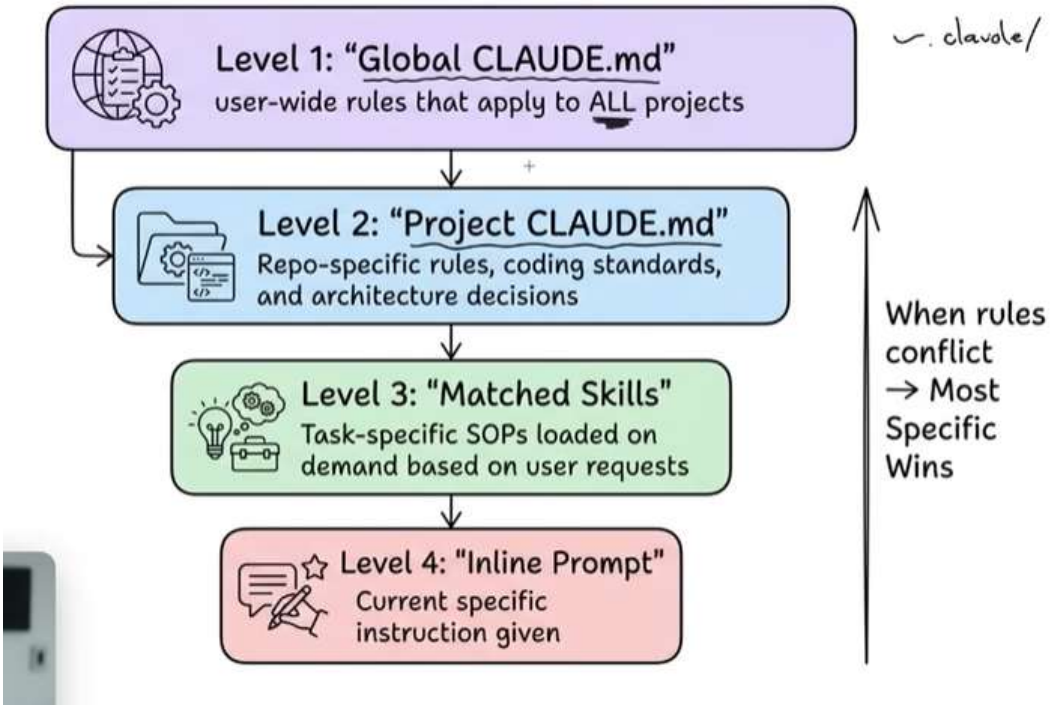
5. Agents.md
6. Self-modifying agents instructions
    a. A
    b. 


SESSION STARTS
AGENT READS GEMINI.MD
CONTAINS ALL PREVIOUS RULES
AGENT WORKS ON TASK
USER CORRECTS A MISTAKE
AGENT WRITES NEW RULE TO GEMINI.MD
e.g. NEVER USE CONST
AGENT CONTINUES WORKING WITH RULE APPLIED
SESSION ENDS
NEXT SESSION STARTS
AGENT GETS PROGRESSIVELY SMARTER AT YOUR CODEBASE

2. K

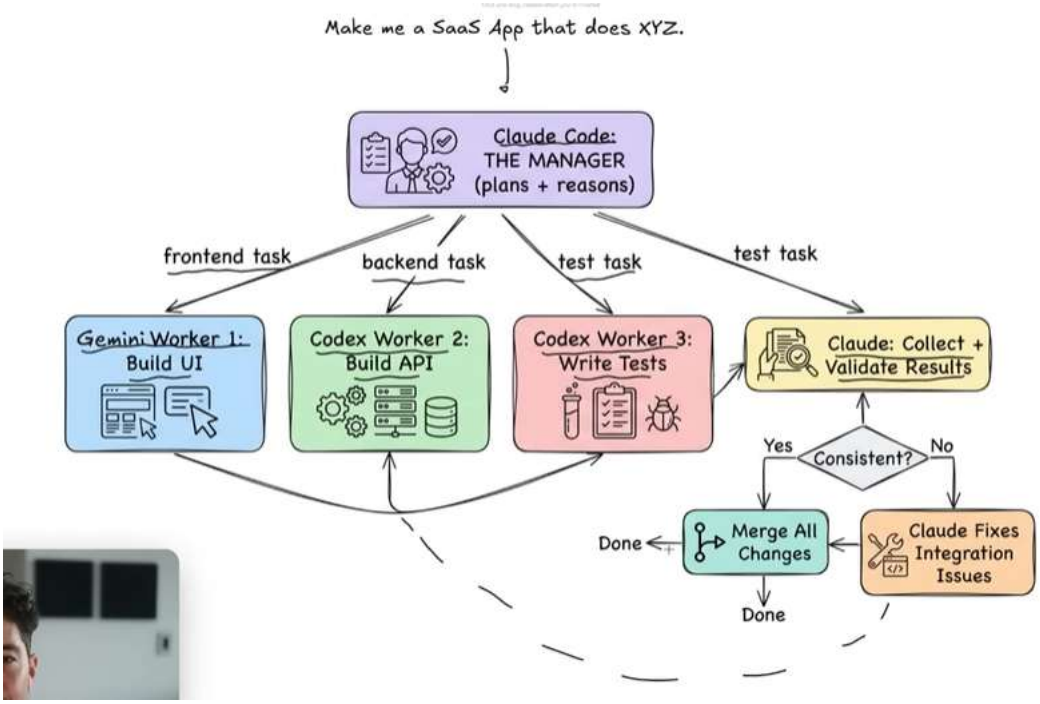

KNOWLEDGE ACCUMULATION OVER SESSIONS
NUMBER
RULES
ERRORS
TIME / SESSIONS
SESSION 1
START: 0 Rules
AGENT MAKES 3 MISTAKES
WRITES 3 RULES
R1 R2 R3
SESSION 2
(Uses 3 Rules)
AVOIDS 3 MISTAKES
MAKES 2 NEW MISTAKES
WRITES 2 RULES
R4 R5
SESSION 3
(Uses 5 Rules)
AVOIDS 5 MISTAKES
MAKES 1 NEW MISTAKE
WRITES 1 RULE
R6
SESSION N
(Uses Many Rules)
AGENT RARELY MAKES MISTAKES
MANY RULES (R1 to RN)

3.

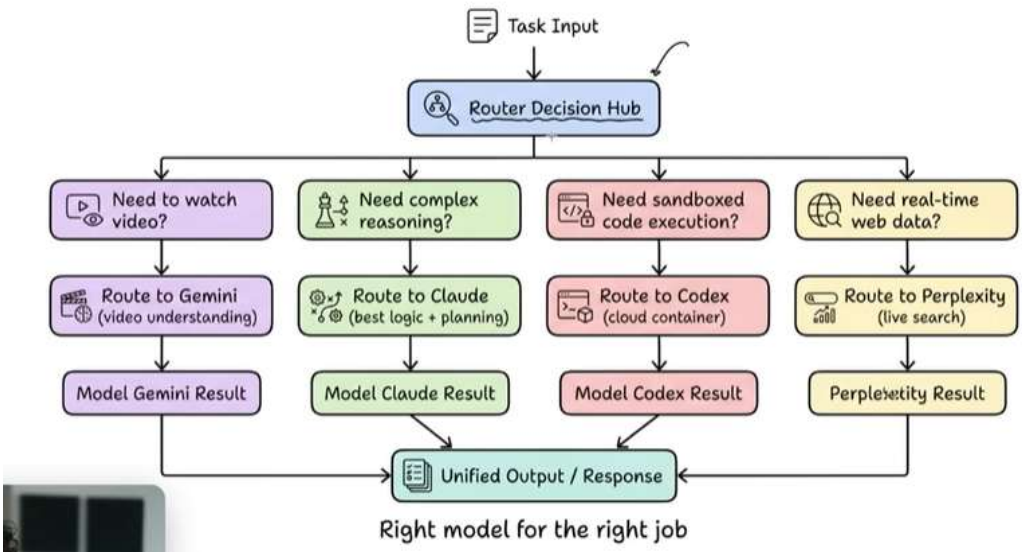


4. Brief walkthrough of Agent skills

7. Multi-agent MCP orchestration (Gemini <> Claude <> GPT)

a. 

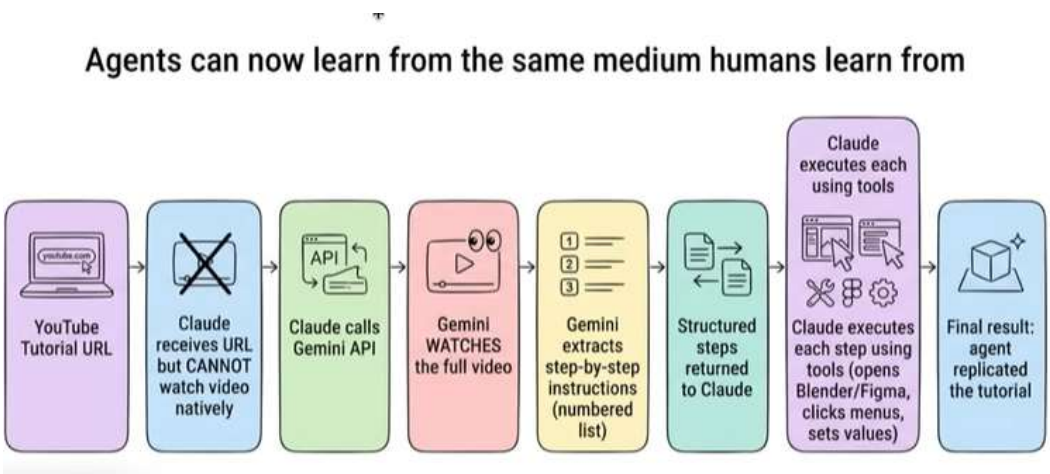
b. All above idea is because of below



3. But above are cosly

8. Video to action pipelines (teaching AI through video)

    a. A



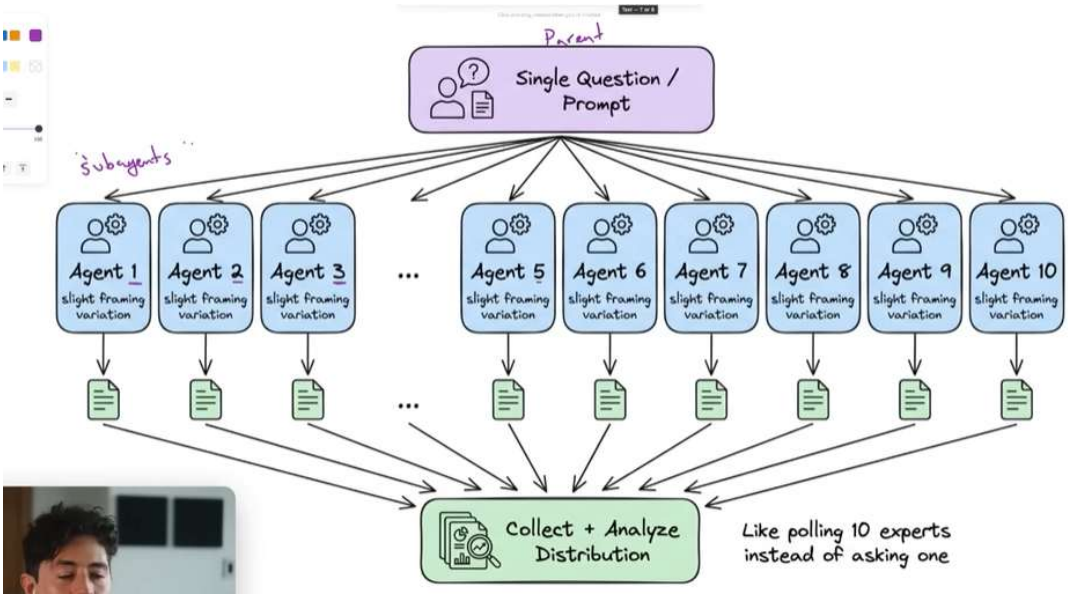
9. Stochastic multi-agent consensus for decision making

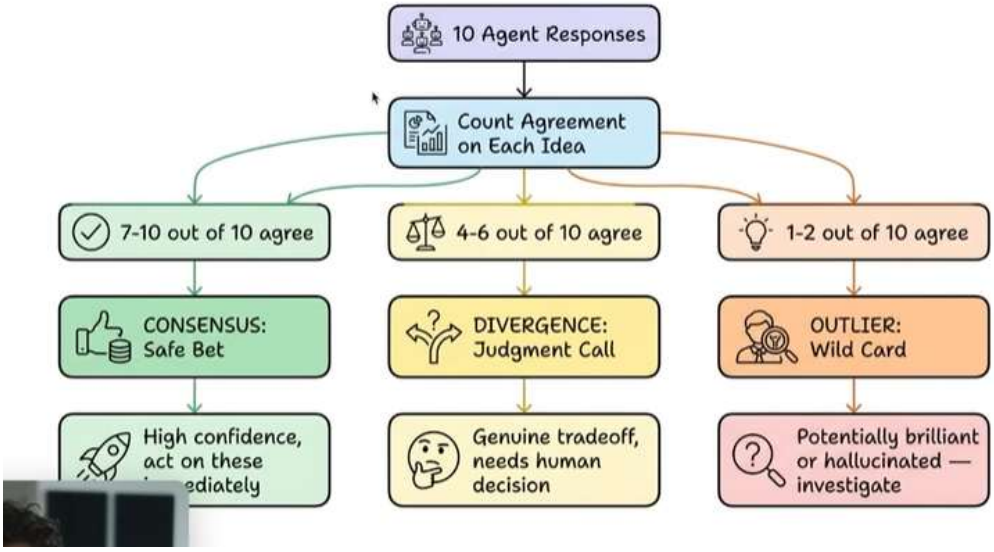
    a. If you ask Gemini .1 Preo - about 10 idea he will do some statical variation in the input or output is different, it will return soe different this property is called stochastic

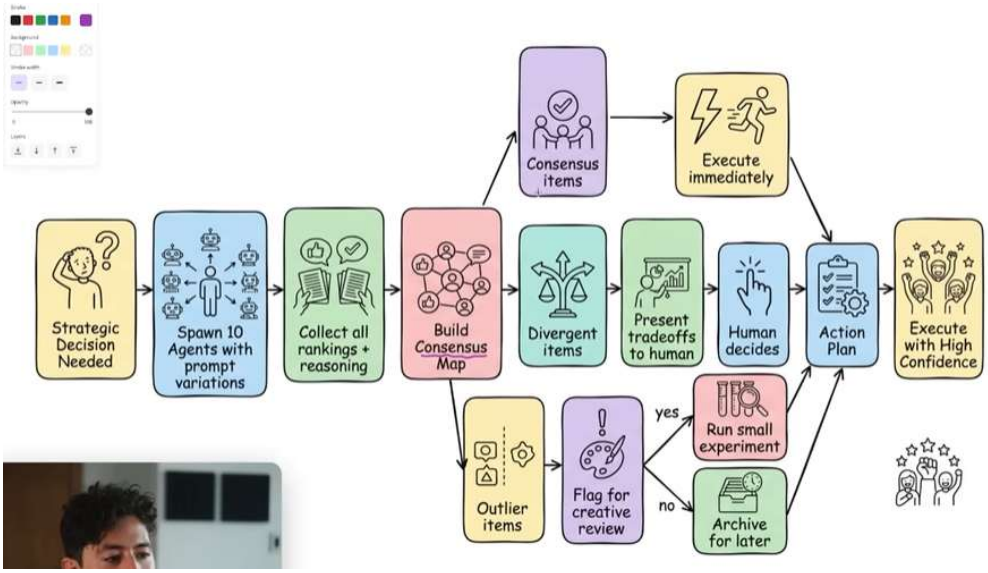
        i. This is valuable

        ii. We get all the possibility; traersingthe search

    b. S

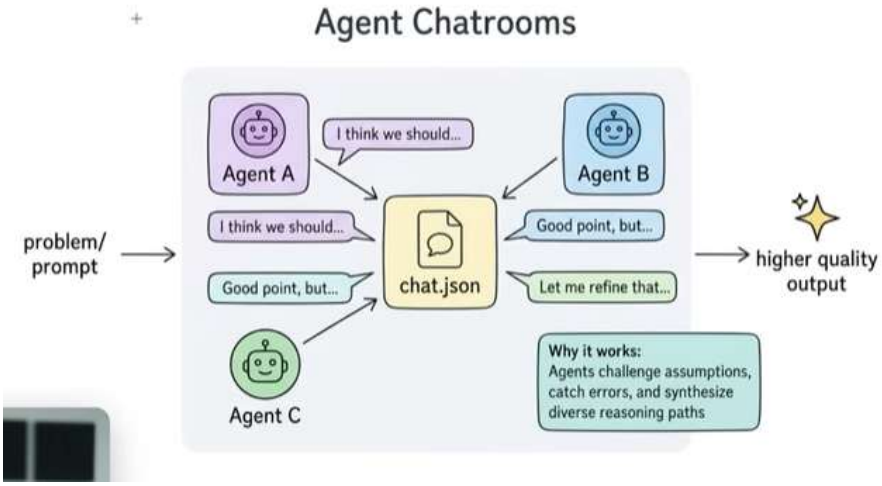

Parent
Single Question / Prompt
subagents
Agent 1
slight framing variation
Agent 2
slight framing variation
Agent 3
slight framing variation
...
Agent 5
slight framing variation
Agent 6
slight framing variation
Agent 7
slight framing variation
Agent 8
slight framing variation
Agent 9
slight framing variation
Agent 10
slight framing variation
Collect + Analyze Distribution
Like polling 10 experts instead of asking one


10 Agent Responses
Count Agreement on Each Idea
7-10 out of 10 agree
4-6 out of 10 agree
1-2 out of 10 agree
CONSENSUS: Safe Bet
DIVERGENCE: Judgment Call
OUTLIER: Wild Card
High confidence, act on these
Genuine tradeoff, needs human decision
Potentially brilliant or hallucinated — investigate



10. Agent chatrooms

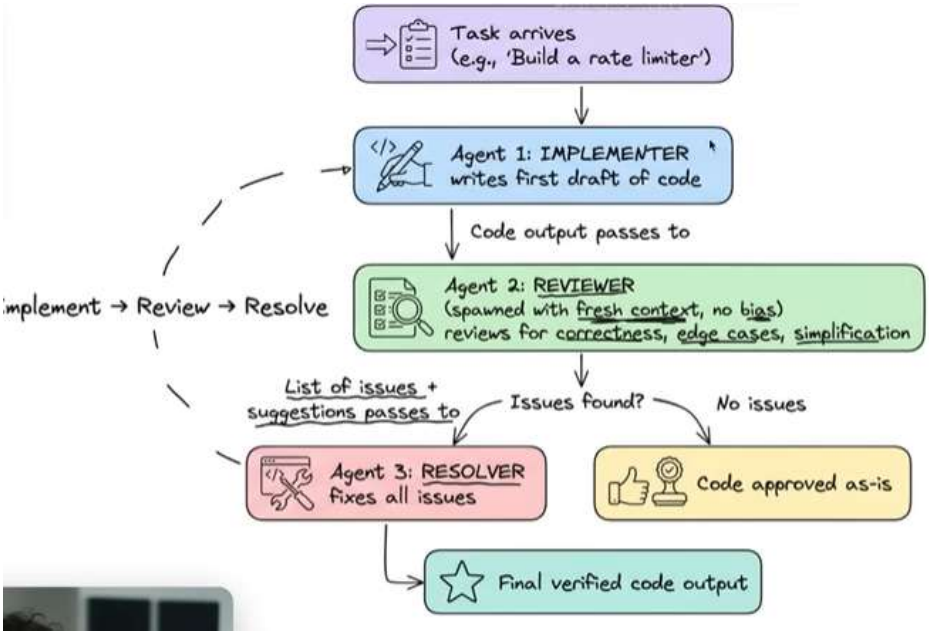
    a. A



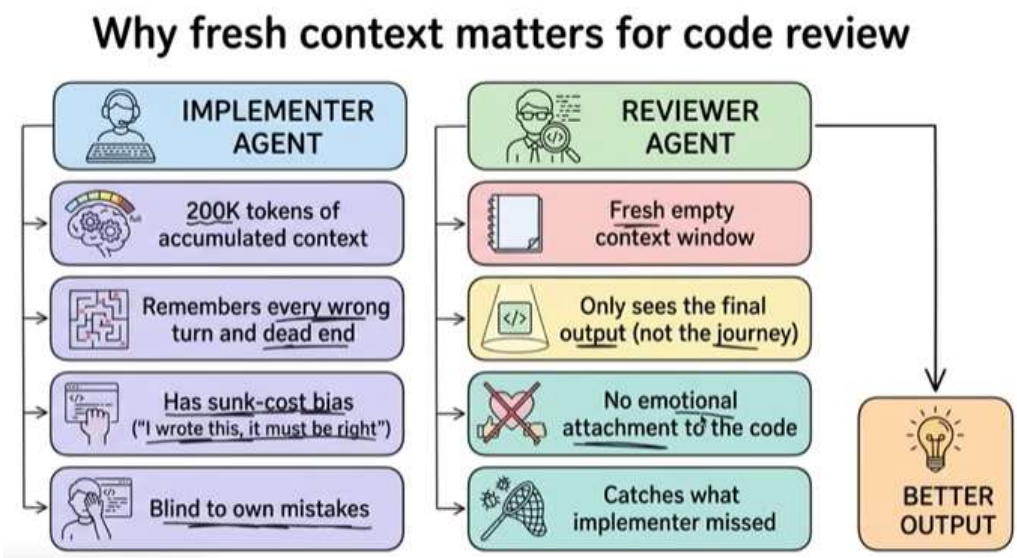
2. A

11. Subagent verification loops: Serial rocessing

    a. A



2. d

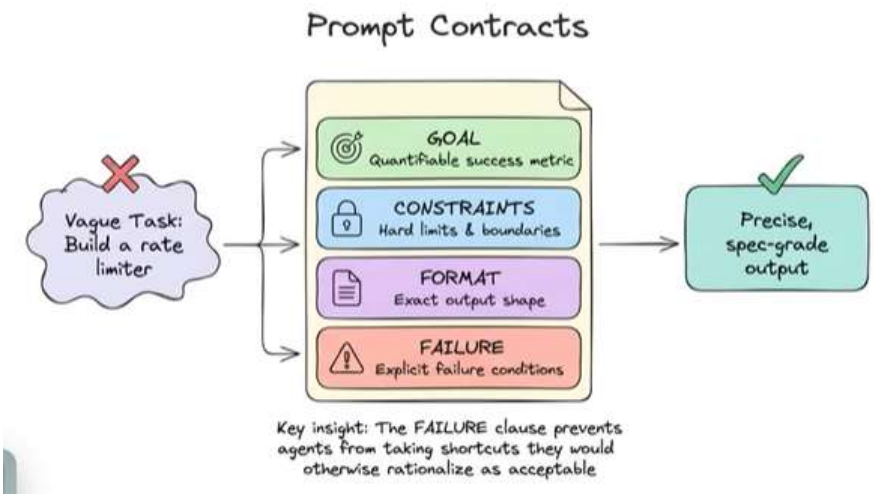


3. C

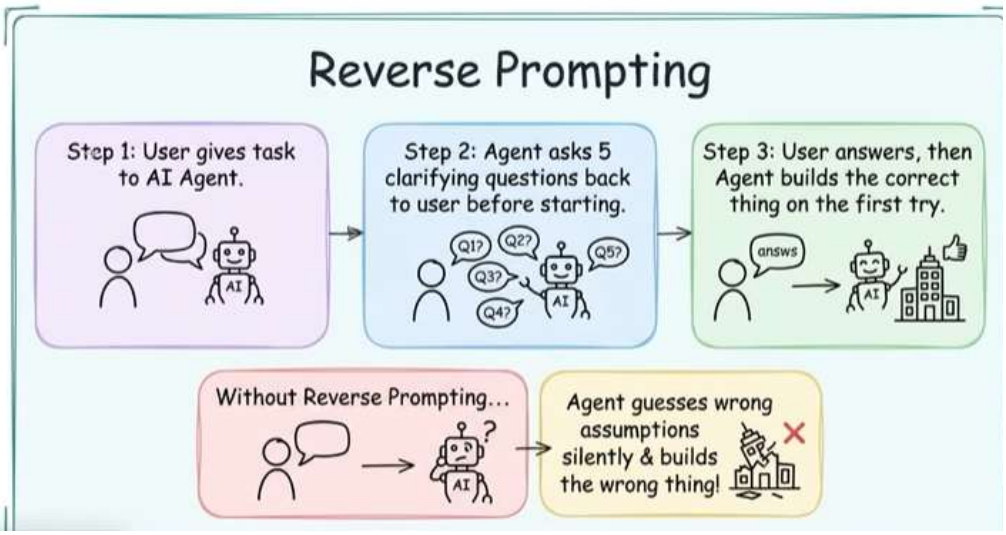
13. Prompt Contracts

    a.

Prompt contracts

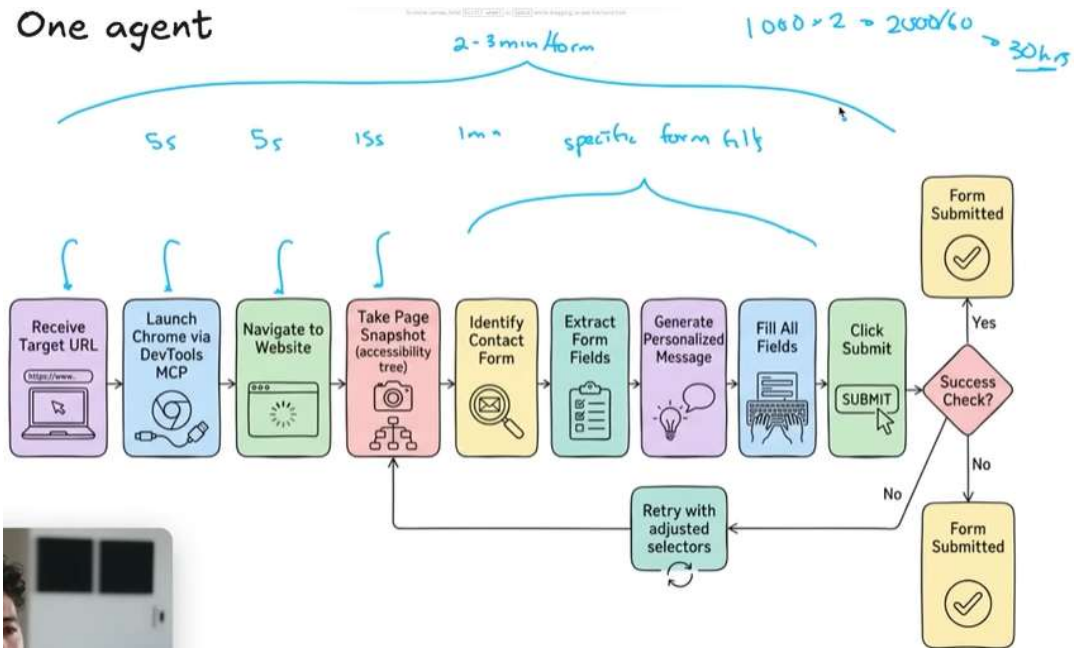


14. Reverse prompting

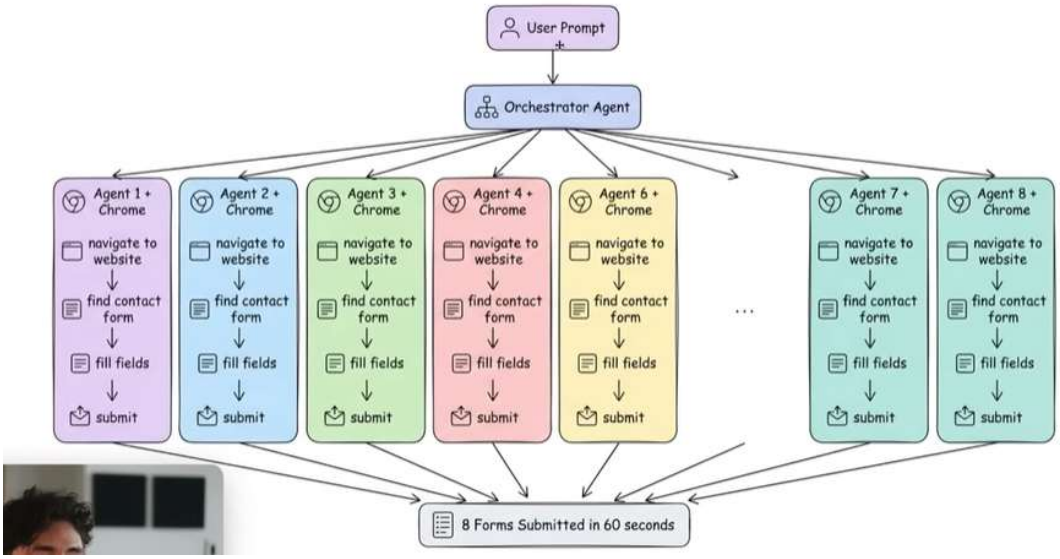
a. 

15. Multi-agent chrome usage

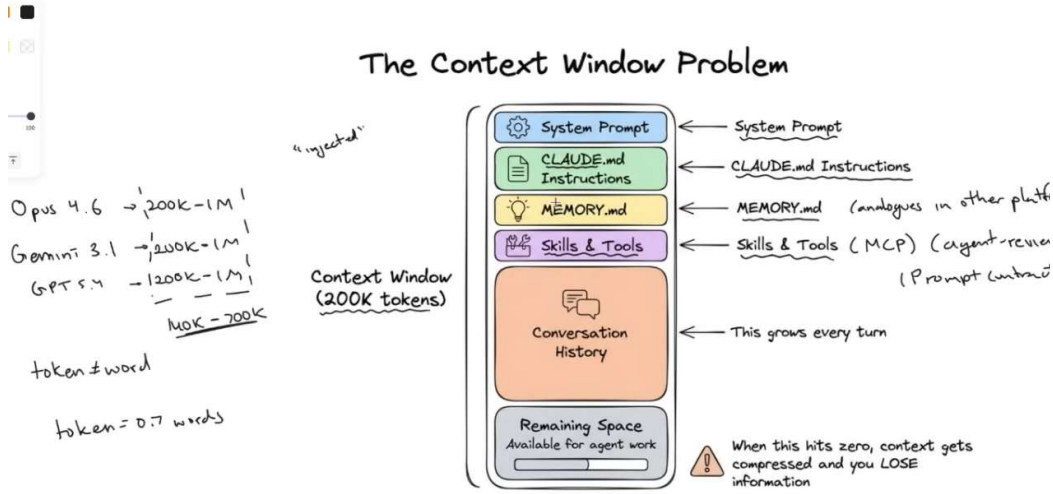
a.



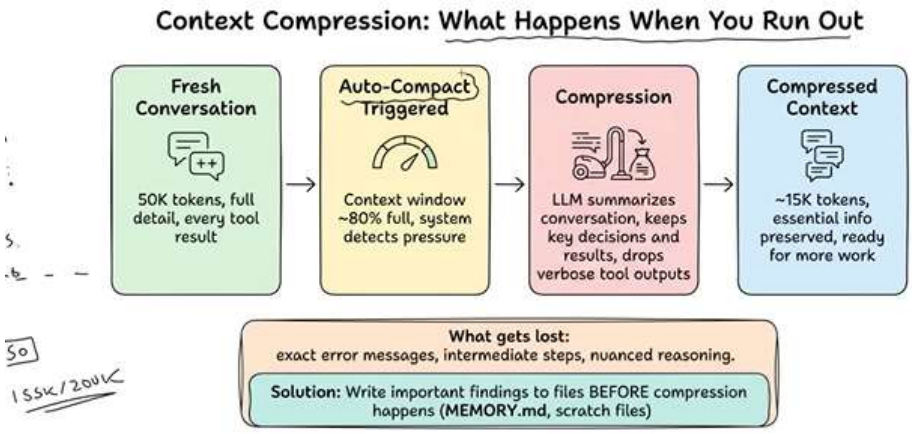
2. A

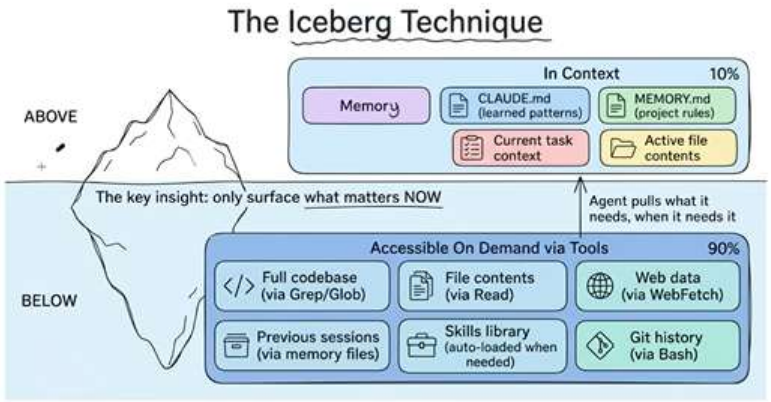


16. Context management

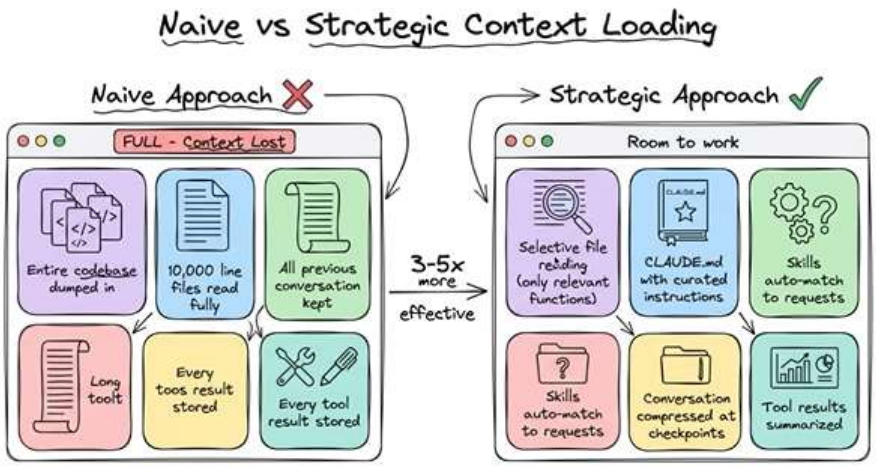

The Context Window Problem
Opus 4.6 → 200K-1M
Gemini 3.1 → 200K-1M
GPT 5.4 → 200K-1M
140K - 200K
token ≠ word
token = 0.7 words
"injected"
Context Window (200K tokens)
System Prompt
CLAUDE.md Instructions
MEMORY.md
Skills & Tools
Conversation History
Remaining Space
Available for agent work
System Prompt
CLAUDE.md Instructions
MEMORY.md
Skills & Tools (MCP)
This grows every turn
When this hits zero, context gets compressed and you LOSE information

a.


Context Compression: What Happens When You Run Out
Fresh Conversation
50K tokens, full detail, every tool result
Auto-Compact Triggered
Context window ~80% full, system detects pressure
Compression
LLM summarizes conversation, keeps key decisions and results, drops verbose tool outputs
Compressed Context
~15K tokens, essential info preserved, ready for more work
What gets lost:
exact error messages, intermediate steps, nuanced reasoning.
Solution: Write important findings to files BEFORE compression happens (MEMORY.md, scratch files)
So
155K/200K

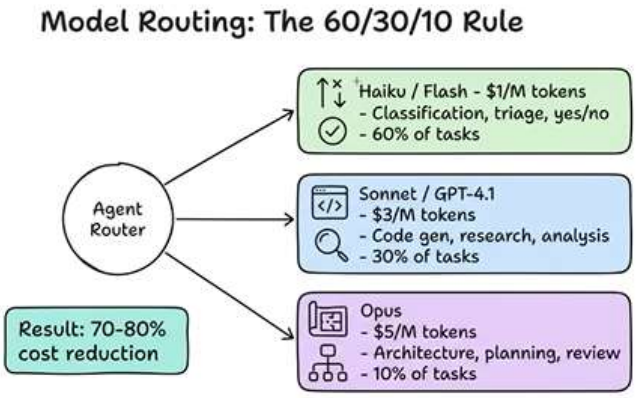

The Iceberg Technique
ABOVE
BELOW
In Context 10%
Memory
CLAUDE.md (learned patterns)
MEMORY.md (project rules)
Current task context
Active file contents
The key insight: only surface what matters NOW
Agent pulls what it needs, when it needs it
Accessible On Demand via Tools 90%
Full codebase (via Grep/Glob)
File contents (via Read)
Web data (via WebFetch)
Previous sessions (via memory files)
Skills library (auto-loaded when needed)
Git history (via Bash)

2.

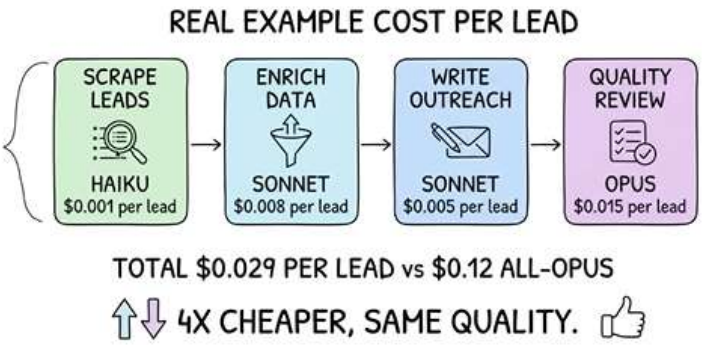


3.

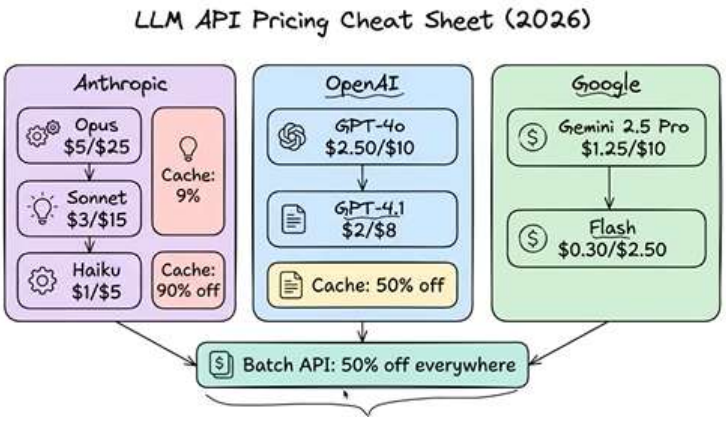
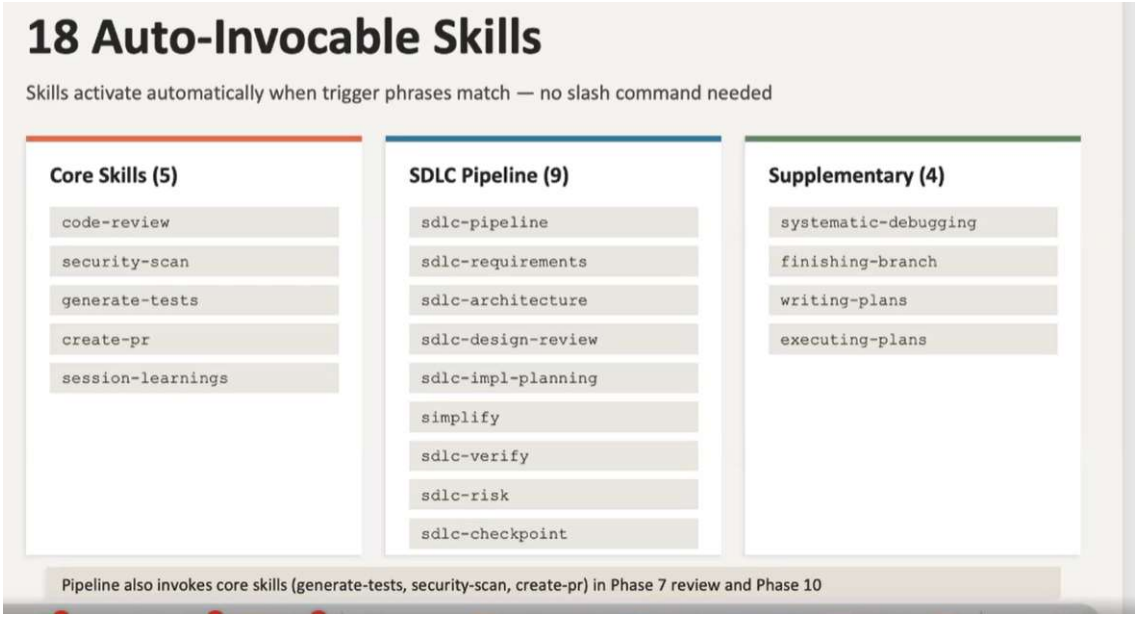
4. Different model for different purpose-- Quality VS Cost



5. Example to save cost



6. Cheat sheet- Company ask you to wait and run when load is less -- batch them

7. 

8.